

Lab Report No 11

Implementation of Image compression II

Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

20th Dec, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Example No 1:

Solution:

Brief description (3-5 lines)

Huffman Coding:

Huffman coding is a lossless compression technique that assigns shorter binary codes to more frequent pixel intensities and longer codes to less frequent ones.

The code

<pre>import cv2 import numpy as np from collections import Counter import matplotlib.pyplot as plt # Load a grayscale image image = cv2.imread("Huffman Coding.jpg", cv2.IMREAD_GRAYSCALE) if image is None: raise FileNotFoundError("Image not found. Please check the file path.") # Calculate pixel intensity frequencies unique, counts = np.unique(image, return_counts=True) frequency = dict(zip(unique, counts)) # Create a Huffman Tree def huffman_tree(frequency): # Create a heap with frequency and symbol pairs heap = [[weight, [symbol, ""]] for symbol, weight in frequency.items()] while len(heap) > 1: heap.sort() # Sort by frequency lo = heap.pop(0) # Remove the two smallest nodes hi = heap.pop(0) for pair in lo[1:]: pair[1] = "0" + pair[1] # Append '0' to the left node for pair in hi[1:]: pair[1] = "1" + pair[1] # Append '1' to the right node # Merge the two nodes and push back to the heap</pre>
--

```

        heap.append([lo[0] + hi[0]] + lo[1:] + hi[1:])
    # Return sorted Huffman codes
    return sorted(heap[0][1:], key=lambda p: (len(p[1]), p))
# Generate Huffman codes
huffman_codes = huffman_tree(frequency)
huffman_dict = {symbol: code for symbol, code in huffman_codes}
# Encode the image
encoded_image = "".join([huffman_dict[pixel] for pixel in
image.flatten()])
# Calculate the size of the Huffman encoded image
original_size = image.size * 8 # Original image size in bits (8 bits per
pixel)
encoded_size = len(encoded_image) # Encoded image size in bits
# Print results
print(f"Original Image Size: {original_size} bits")
print(f"Huffman Encoded Image Size: {encoded_size} bits")
# Save Huffman codes to a file (optional)
with open("huffman_codes.txt", "w") as f:
    for symbol, code in huffman_codes:
        f.write(f'{symbol}: {code}\n')
# Save the encoded image (optional)
with open("encoded_image.txt", "w") as f:
    f.write(encoded_image)
# Display the original image
plt.figure(figsize=(8, 6))
plt.imshow(image, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()

```

The results (Screenshot)

```

Original Image Size: 11520000 bits
Huffman Encoded Image Size: 9206280 bits
Original Image Size: 11520000 bits
Huffman Encoded Image Size: 9206280 bits
Huffman Encoded Image Size: 9206280 bits

```

Original Grayscale Image



HUFFMAN CODES PIXEL OF THE PICTURE

15: 00011
16: 01110
17: 00101
19: 01111
20: 10010
21: 10001
22: 10111
23: 11011
24: 01000
26: 01100
27: 01001
28: 00001
8: 001110

9: 111101
10: 110101
11: 001000
12: 101010
13: 111100
14: 111110
18: 001100
25: 000101
29: 111010
30: 110000
31: 100000
33: 100110
34: 101100
35: 110001
36: 101001
37: 011010
41: 000001
3: 0101011
4: 0101101
5: 0101111
6: 1000010
7: 1100111
32: 1110010
38: 1111110
39: 1100100
40: 1100110
42: 1110110
43: 1101001
44: 1010111
45: 0110110
46: 0011111
48: 0110111
49: 1010000
50: 1001110
51: 0101100

52: 0000000
55: 0010010
56: 0101010
57: 0011010
47: 11111111
53: 11100010
54: 11100000
58: 11111110
59: 10110111
60: 11001011
61: 10100010
62: 11100001
63: 11100110
64: 11010001
65: 10101101
66: 00110111
67: 00111100
69: 00100111
70: 01010011
71: 01010000
72: 00110110
77: 00010001
78: 00000011
2: 101101010
68: 110100001
73: 111011101
74: 111011111
75: 100001100
76: 111001111
79: 111000111
80: 101011000
81: 100001111
82: 001001101
83: 010111000
84: 101101101

85: 101101100
86: 101000111
87: 100111100
88: 010100011
90: 000000100
91: 010111001
92: 010100100
93: 000100001
94: 000100000
99: 000100111
100: 000100100
89: 1110111000
95: 1100101011
96: 1000011011
97: 1011010000
98: 1110111101
101: 1100101001
102: 1001111010
103: 0101110101
104: 1000011100
105: 1011010011
106: 1101000001
107: 1010110010
108: 1001111111
109: 0101000101
112: 1000011101
113: 0101000100
114: 0001001100
119: 0011110111
120: 0011110100
121: 0010011000
142: 0001001010
110: 11100011011
111: 11001010101
115: 11101110011

116: 11010000001
117: 10110100010
118: 10100011000
122: 11101111000
123: 10110101100
124: 10011111011
125: 10000110101
126: 11100011010
127: 11101111001
128: 11101110010
129: 11100111011
130: 01011101000
131: 10100011011
132: 01011101101
133: 10110101111
134: 11001010001
135: 11001010000
136: 10110100101
137: 01010010110
138: 01010010101
139: 10110100011
140: 11100111000
141: 11100111001
143: 10100011010
144: 00100110010
145: 00111101011
146: 00010011010
147: 01011101111
148: 10101100110
149: 10110101110
150: 10011111101
151: 01011101001
152: 01011101110
153: 00111101010
154: 10011111001

155: 11001010100
156: 10011111000
161: 00000010111
162: 00010010110
163: 00000010101
1: 111000110001
157: 111000110010
158: 101101001001
159: 110100000000
160: 101000110010
164: 101000110011
165: 100111101101
166: 100111110100
167: 010100101111
168: 000100110110
169: 101101011010
170: 100111110101
171: 000100101110
176: 000000101001
172: 1110001100000
173: 1001111011110
174: 1010110011101
175: 1000011010000
177: 1110001100111
178: 1011010010001
179: 1001111011111
180: 0011110110111
181: 0000001011010
182: 0011110110101
183: 1000011010011
184: 1001111011100
185: 1010110011110
186: 0101110110010
187: 0101110110000
188: 0000001011001

189: 0101001010000
190: 1001111110000
191: 1001111110001
192: 1010110011100
193: 0101001010010
194: 0001001101111
197: 0011110110010
198: 0010011001100
199: 0101001010001
201: 0101110110011
204: 1001111011000
205: 0101001011101
206: 0011110110011
208: 0000001010000
209: 0001001011111
211: 0011110110000
212: 1000011010001
213: 1001111011001
215: 0101001011100
216: 0011110110100
218: 0010011001110
219: 0001001101110
220: 0000001010001
221: 0000001011000
226: 0011110110110
227: 0010011001111
232: 0000001011011
233: 0011110110001
234: 0101001010011
235: 1001111011101
237: 0001001011110
240: 1001111110010
241: 1001111110011
242: 1011010110110
243: 1000011010010

244: 1011010010000
195: 11010000000110
196: 11010000000100
200: 11100011001100
202: 11100111010000
203: 11100111010100
207: 11100111010101
210: 10110101101110
214: 11100011001101
217: 01011101100010
222: 10110101101111
223: 10101100111110
225: 11100111010001
228: 11100011000010
229: 11100111010011
230: 10101100111111
236: 11100011000011
239: 11100111010111
245: 11010000000101
246: 11100111010110
248: 00100110011011
224: 111001110100101
231: 110100000001110
238: 110100000001111
247: 010111011000111
250: 001001100110101
251: 001001100110100
0: 1110011101001000
249: 1110011101001001
252: 0101110110001101
253: 01011101100011001
254: 010111011000110001
255: 010111011000110000

Lab Example No 2:

Solution:

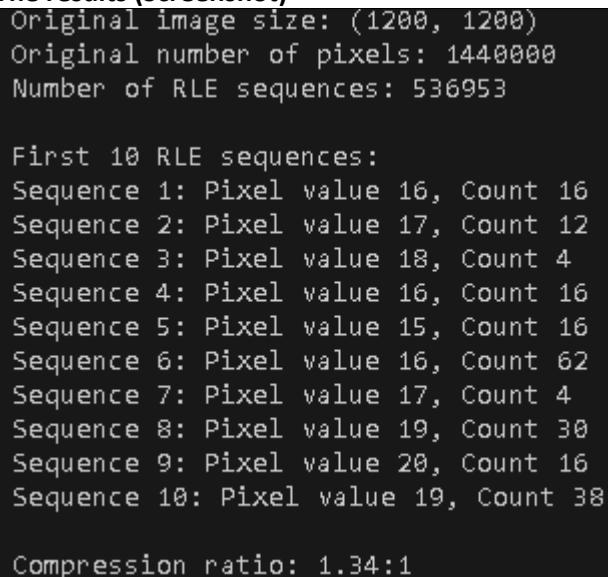
Brief description (3-5 lines)
Run-Length Coding (RLC): RLC compresses the consecutive identical pixel values into a value and its count.
The code
<pre>import cv2 import numpy as np def run_length_encode(data): encoding = [] prev_pixel = data[0] count = 1 # Iterate through the array starting from the second element for pixel in data[1:]: if pixel == prev_pixel: count += 1 else: encoding.append((prev_pixel, count)) prev_pixel = pixel count = 1 # Append the last sequence encoding.append((prev_pixel, count)) return encoding def main(): # Read the image image = cv2.imread('Run-Length Coding.jpg', cv2.IMREAD_GRAYSCALE) # Read as grayscale # Flatten the image into a 1D array flat_image = image.flatten() # Perform RLE</pre>

```

encoded_rlc = run_length_encode(flat_image)
# Print some statistics
print(f'Original image size: {image.shape}')
print(f'Original number of pixels: {len(flat_image)}')
print(f'Number of RLE sequences: {len(encoded_rlc)}')
# Print first few encoded sequences
print("\nFirst 10 RLE sequences:")
for i, (pixel, count) in enumerate(encoded_rlc[:10]):
    print(f'Sequence {i+1}: Pixel value {pixel}, Count {count}')
# Calculate compression ratio
original_size = len(flat_image)
compressed_size = len(encoded_rlc) * 2 # Each tuple has 2
numbers (pixel_value, count)
compression_ratio = original_size / compressed_size
print(f'\nCompression ratio: {compression_ratio:.2f}:1')
if __name__ == "__main__":
    main()

```

The results (Screenshot)



```

Original image size: (1200, 1200)
Original number of pixels: 1440000
Number of RLE sequences: 536953

First 10 RLE sequences:
Sequence 1: Pixel value 16, Count 16
Sequence 2: Pixel value 17, Count 12
Sequence 3: Pixel value 18, Count 4
Sequence 4: Pixel value 16, Count 16
Sequence 5: Pixel value 15, Count 16
Sequence 6: Pixel value 16, Count 62
Sequence 7: Pixel value 17, Count 4
Sequence 8: Pixel value 19, Count 30
Sequence 9: Pixel value 20, Count 16
Sequence 10: Pixel value 19, Count 38

Compression ratio: 1.34:1

```

Lab Example No 3:

Solution:

Brief description (3-5 lines)

Bit Plane Coding:

Bit plane coding partitions an image into distinct binary planes that represent each of the bits of a pixel intensity as a binary image.

The code

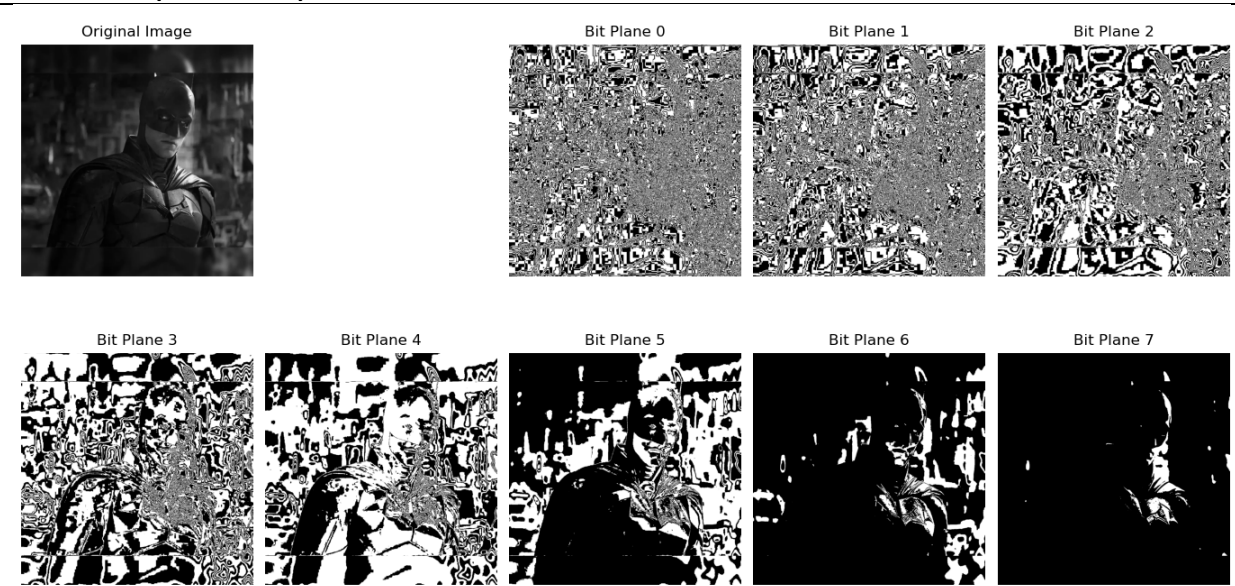
```
import cv2
import matplotlib.pyplot as plt
import numpy as np
def bit_plane_slicing(image):
    # Get image dimensions
    rows, cols = image.shape
    # Initialize list to store bit planes
    bit_planes = []
    # Extract each bit plane
    for bit in range(8):
        # Use bitwise AND with 1 shifted left by 'bit' positions
        # Then shift right by 'bit' positions to get binary image
        bit_plane = (image & (1 << bit)) >> bit
        bit_planes.append(bit_plane)
    return bit_planes
def display_bit_planes(image_path):
    # Read the image in grayscale
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    # Check if image was successfully loaded
    if image is None:
        print(f'Error: Could not load image from {image_path}')
        return
    # Get bit planes
    bit_planes = bit_plane_slicing(image)
    # Create figure for displaying results
    plt.figure(figsize=(15, 8))
    # Display original image
    plt.subplot(2, 5, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
```

```

# Display each bit plane
for i, plane in enumerate(bit_planes):
    plt.subplot(2, 5, i + 3)
    plt.imshow(plane, cmap='gray')
    plt.title(f'Bit Plane {i}')
    plt.axis('off')
# Adjust layout and display
plt.tight_layout()
plt.show()
# Example usage
if __name__ == "__main__":
    # Replace with your image path
    image_path = "Bit Plane Coding.jpg"
    display_bit_planes(image_path)

```

The results (Screenshot)



Lab Task No 1:

Solution:

Brief description (3-5 lines)

Carry out Huffman coding on a grayscale image and analyze the efficiency of compression.

The code

```
import numpy as np
from collections import defaultdict
import heapq
from PIL import Image
import os
class HuffmanNode:
    def __init__(self, pixel=None, freq=None):
        self.pixel = pixel
        self.freq = freq
        self.left = None
        self.right = None
    def __lt__(self, other):
        return self.freq < other.freq
class HuffmanImageCompression:
    def __init__(self):
        self.codes = {}
        self.reverse_codes = {}
    def calculate_frequency(self, image_array):
        """Calculate frequency of each pixel value in the image."""
        frequency = defaultdict(int)
        for pixel in image_array.flatten():
            frequency[pixel] += 1
        return frequency
    def build_huffman_tree(self, frequency):
        """Build Huffman tree using priority queue."""
        priority_queue = []
        # Create leaf nodes for each pixel value
        for pixel, freq in frequency.items():
            node = HuffmanNode(pixel, freq)
            heapq.heappush(priority_queue, node)
        # Build the tree
        while len(priority_queue) > 1:
            left = heapq.heappop(priority_queue)
            right = heapq.heappop(priority_queue)
```



```

        internal_node = HuffmanNode()
        internal_node.freq = left.freq + right.freq
        internal_node.left = left
        internal_node.right = right
        heapq.heappush(priority_queue, internal_node)
    return priority_queue[0]
def generate_codes(self, root, code=""):
    """Generate Huffman codes for each pixel value."""
    if root is None:
        return
    if root.pixel is not None:
        self.codes[root.pixel] = code
        self.reverse_codes[code] = root.pixel
        return
    self.generate_codes(root.left, code + "0")
    self.generate_codes(root.right, code + "1")
def compress(self, image_path):
    """Compress the image using Huffman coding."""
    # Read image
    image = Image.open(image_path).convert('L') # Convert to
    grayscale
    image_array = np.array(image)
    # Calculate frequencies
    frequency = self.calculate_frequency(image_array)
    # Build Huffman tree and generate codes
    root = self.build_huffman_tree(frequency)
    self.generate_codes(root)
    # Encode image
    encoded_image = ""
    for pixel in image_array.flatten():
        encoded_image += self.codes[pixel]
    # Calculate compression statistics
    original_size = image_array.size * 8 # 8 bits per pixel
    compressed_size = len(encoded_image)
    compression_ratio = original_size / compressed_size

```

```

return {
    'encoded_data': encoded_image,
    'codes': self.codes,
    'original_size': original_size,
    'compressed_size': compressed_size,
    'compression_ratio': compression_ratio
}

def decompress(self, encoded_data, shape):
    """Decompress the encoded image data."""
    current_code = ""
    decoded_pixels = []
    for bit in encoded_data:
        current_code += bit
        if current_code in self.reverse_codes:
            decoded_pixels.append(self.reverse_codes[current_code])
            current_code = ""
    # Reshape back to original image dimensions
    decoded_array = np.array(decoded_pixels,
dtype=np.uint8).reshape(shape)
    return decoded_array

# Example usage and analysis
def analyze_compression(image_path):
    """Analyze the compression efficiency for a given image."""
    compressor = HuffmanImageCompression()
    # Compress the image
    compression_results = compressor.compress(image_path)
    # Calculate compression statistics
    original_size_kb = compression_results['original_size'] / 8000 #
Convert bits to KB
    compressed_size_kb = compression_results['compressed_size'] /
8000
    compression_ratio = compression_results['compression_ratio']
    space_saving = (1 - 1/compression_ratio) * 100
    # Print analysis

```

```

print(f'\nCompression Analysis for
{os.path.basename(image_path)}:')
print(f'Original Size: {original_size_kb:.2f} KB")
print(f'Compressed Size: {compressed_size_kb:.2f} KB")
print(f'Compression Ratio: {compression_ratio:.2f}:1")
print(f'Space Saving: {space_saving:.2f}%")
return compression_results
# Usage example:
if __name__ == "__main__":
    # Replace with your image path
    image_path = "huffmancoding.jpg"
    results = analyze_compression(image_path)

```

The results (Screenshot)

```

Compression Analysis for huffmancoding.jpg:
Original Size: 1440.00 KB
Compressed Size: 1150.79 KB
Compression Ratio: 1.25:1
Space Saving: 20.08%

```

Lab Task No 2:

Solution:

Brief description (3-5 lines)

Apply different coding to the same image and observe how it can reduce redundancy.

The code

```

import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from collections import Counter
import math
class DifferenceCoding:
    def __init__(self):
        self.original_entropy = None
        self.encoded_entropy = None

```

```

def encode(self, image_array):
    """Apply difference coding to the image."""
    height, width = image_array.shape
    encoded = np.zeros_like(image_array, dtype=np.int16)
    # First pixel remains unchanged
    encoded[0, 0] = image_array[0, 0]
    # Encode first row
    for j in range(1, width):
        encoded[0, j] = int(image_array[0, j]) - int(image_array[0, j-1])
    # Encode first column
    for i in range(1, height):
        encoded[i, 0] = int(image_array[i, 0]) - int(image_array[i-1, 0])
    # Encode rest of the image
    # Using average of left and top pixels as predictor
    for i in range(1, height):
        for j in range(1, width):
            predictor = (int(image_array[i-1, j]) + int(image_array[i, j-1])) // 2
            encoded[i, j] = int(image_array[i, j]) - predictor
    return encoded

def decode(self, encoded_array):
    """Decode the difference coded image."""
    height, width = encoded_array.shape
    decoded = np.zeros_like(encoded_array, dtype=np.uint8)
    # First pixel remains unchanged
    decoded[0, 0] = encoded_array[0, 0]
    # Decode first row
    for j in range(1, width):
        decoded[0, j] = encoded_array[0, j] + decoded[0, j-1]
    # Decode first column
    for i in range(1, height):
        decoded[i, 0] = encoded_array[i, 0] + decoded[i-1, 0]
    # Decode rest of the image
    for i in range(1, height):
        for j in range(1, width):

```

```

        predictor = (int(decoded[i-1, j]) + int(decoded[i, j-1])) // 2
        decoded[i, j] = encoded_array[i, j] + predictor
    return decoded
def calculate_entropy(self, array):
    """Calculate entropy of the array."""
    counter = Counter(array.flatten())
    total_pixels = array.size
    entropy = 0
    for count in counter.values():
        probability = count / total_pixels
        entropy -= probability * math.log2(probability)
    return entropy
def analyze_redundancy(self, image_path):
    """Analyze redundancy reduction after difference coding."""
    # Read and convert image to grayscale
    image = Image.open(image_path).convert('L')
    image_array = np.array(image)
    # Apply difference coding
    encoded_array = self.encode(image_array)
    # Calculate entropies
    self.original_entropy = self.calculate_entropy(image_array)
    self.encoded_entropy = self.calculate_entropy(encoded_array)
    # Calculate redundancy reduction
    redundancy_reduction = ((self.original_entropy -
self.encoded_entropy) /
                           self.original_entropy * 100)
    # Create visualization
    plt.figure(figsize=(15, 5))
    # Original image histogram
    plt.subplot(131)
    plt.hist(image_array.flatten(), bins=256, range=(0, 256),
density=True)
    plt.title('Original Image Histogram')
    plt.xlabel('Pixel Value')
    plt.ylabel('Frequency')

```

```

# Difference coded histogram
plt.subplot(132)
plt.hist(encoded_array.flatten(), bins=256, density=True)
plt.title('Difference Coded Histogram')
plt.xlabel('Difference Value')
plt.ylabel('Frequency')
# Decoded image
decoded_array = self.decode(encoded_array)
plt.subplot(133)
plt.imshow(decoded_array, cmap='gray')
plt.title('Decoded Image')
plt.axis('off')
plt.tight_layout()
# Print analysis results
print("\nRedundancy Analysis:")
print(f'Original Entropy: {self.original_entropy:.2f} bits/pixel')
print(f'Encoded Entropy: {self.encoded_entropy:.2f} bits/pixel')
print(f'Redundancy Reduction: {redundancy_reduction:.2f}%')
# Calculate value range statistics
original_range = np.ptp(image_array)
encoded_range = np.ptp(encoded_array)
print(f"\nValue Range Analysis:")
print(f'Original Range: {original_range}')
print(f'Encoded Range: {encoded_range}')
# Calculate standard deviations
original_std = np.std(image_array)
encoded_std = np.std(encoded_array)
print(f"\nStandard Deviation Analysis:")
print(f'Original Std Dev: {original_std:.2f}')
print(f'Encoded Std Dev: {encoded_std:.2f}')
return {
    'original_entropy': self.original_entropy,
    'encoded_entropy': self.encoded_entropy,
    'redundancy_reduction': redundancy_reduction,
    'original_array': image_array,

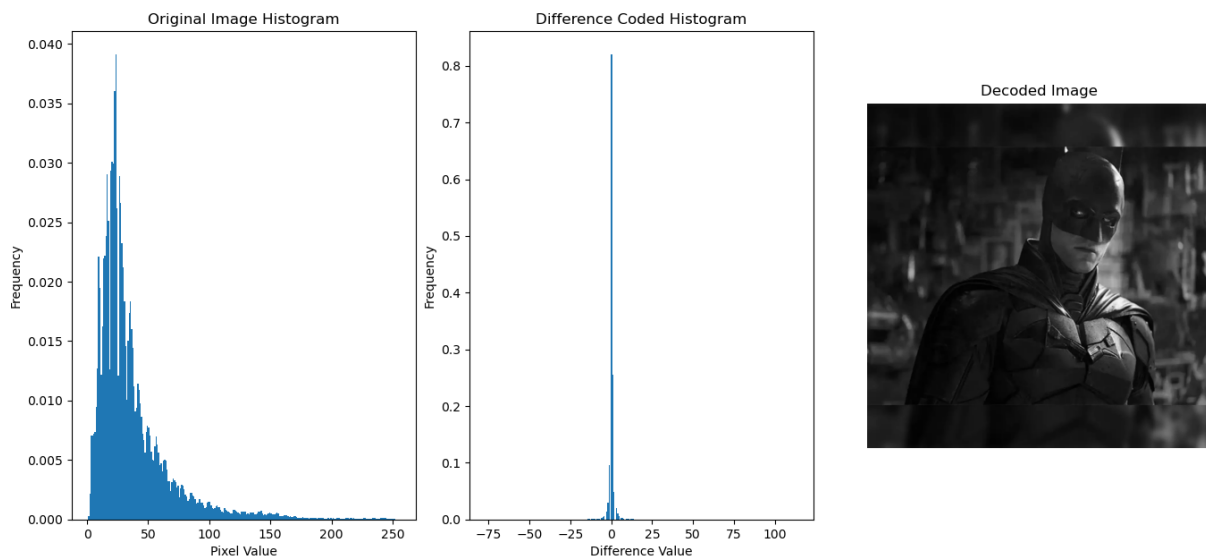
```

```

        'encoded_array': encoded_array,
        'decoded_array': decoded_array
    }
}
# Example usage
if __name__ == "__main__":
    # Replace with your image path
    image_path = "differencecoding.jpg"
    difference_coder = DifferenceCoding()
    results = difference_coder.analyze_redundancy(image_path)
    plt.show()

```

The results (Screenshot)



```

Redundancy Analysis:
Original Entropy: 6.36 bits/pixel
Encoded Entropy: 2.02 bits/pixel
Redundancy Reduction: 68.31%

```

```

Value Range Analysis:
Original Range: 255
Encoded Range: 191

```

```

Standard Deviation Analysis:
Original Std Dev: 31.47
Encoded Std Dev: 2.54

```

Lab Task No 3:

Solution:

Brief description (3-5 lines)
Apply run-length encoding to generate a binary representation of the image.
The code
<pre>import numpy as np from PIL import Image import matplotlib.pyplot as plt from typing import List, Tuple import time class BinaryRLC: def __init__(self, threshold: int = 127): self.threshold = threshold def convert_to_binary(self, image_path: str) -> np.ndarray: """Convert grayscale image to binary.""" # Read image and convert to grayscale image = Image.open(image_path).convert('L') # Convert to numpy array and threshold return (np.array(image) > self.threshold).astype(np.uint8) def run_length_encode(self, binary_image: np.ndarray) -> List[int]: """Perform run-length coding on binary image.""" # Flatten the image flat_image = binary_image.flatten() # Initialize variables encoded = [] count = 1 current = flat_image[0] # Count runs for pixel in flat_image[1:]: if pixel == current: count += 1 else:</pre>


```

        encoded.append(count)
        current = pixel
        count = 1
    # Append the last run
    encoded.append(count)
    return encoded
def run_length_decode(self, encoded: List[int], shape: Tuple[int,
int],
                        first_value: int = 0) -> np.ndarray:
    """Decode RLC data back to binary image."""
    # Initialize output array
    decoded = np.zeros(shape[0] * shape[1], dtype=np.uint8)
    # Track position and current value
    pos = 0
    value = first_value
    # Reconstruct the image
    for run_length in encoded:
        decoded[pos:pos + run_length] = value
        pos += run_length
        value = 1 - value # Toggle between 0 and 1
    return decoded.reshape(shape)
def analyze_compression(self, image_path: str) -> dict:
    """Perform compression analysis with visualizations."""
    # Start timing
    start_time = time.time()
    # Convert to binary
    binary_image = self.convert_to_binary(image_path)
    # Encode the image
    encoded_data = self.run_length_encode(binary_image)
    # Calculate first value for decoding
    first_value = binary_image.flatten()[0]
    # Decode the image
    decoded_image = self.run_length_decode(encoded_data,
binary_image.shape, first_value)
    # End timing

```

```

process_time = time.time() - start_time
# Calculate compression metrics
original_size = binary_image.size // 8 # Size in bytes (8 bits per
byte)
encoded_size = len(encoded_data) * 4 # Size in bytes (32-bit
integers)
compression_ratio = original_size / encoded_size
# Create visualization
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
# Original grayscale image
original_image = Image.open(image_path).convert('L')
axes[0, 0].imshow(original_image, cmap='gray')
axes[0, 0].set_title('Original Grayscale')
axes[0, 0].axis('off')
# Binary image
axes[0, 1].imshow(binary_image, cmap='binary')
axes[0, 1].set_title('Binary Image')
axes[0, 1].axis('off')
# Decoded image
axes[1, 0].imshow(decoded_image, cmap='binary')
axes[1, 0].set_title('Decoded Image')
axes[1, 0].axis('off')
# Run length distribution
axes[1, 1].hist(encoded_data, bins=50, color='blue', alpha=0.7)
axes[1, 1].set_title('Run Length Distribution')
axes[1, 1].set_xlabel('Run Length')
axes[1, 1].set_ylabel('Frequency')
plt.tight_layout()
# Print analysis
print("\nRun-Length Coding Analysis:")
print("-" * 30)
print(f'Original Size: {original_size:,} bytes')
print(f'Encoded Size: {encoded_size:,} bytes')
print(f'Compression Ratio: {compression_ratio:.2f}:1")
print(f'Space Saving: {(1 - 1/compression_ratio) * 100:.2f}%")

```

```

print(f'Processing Time: {process_time:.3f} seconds")
# Run length statistics
print("\nRun Length Statistics:")
print("-" * 30)
print(f'Number of Runs: {len(encoded_data):,}')
print(f'Average Run Length: {np.mean(encoded_data):.2f}')
print(f'Maximum Run Length: {np.max(encoded_data):,}')
print(f'Minimum Run Length: {np.min(encoded_data):,}')
# Verify compression accuracy
compression_accurate = np.array_equal(binary_image,
decoded_image)
print("\nCompression Verification:")
print("-" * 30)
print(f'Lossless Compression: {'Yes' if compression_accurate
else 'No'})
return {
    'binary_image': binary_image,
    'encoded_data': encoded_data,
    'decoded_image': decoded_image,
    'compression_ratio': compression_ratio,
    'original_size': original_size,
    'encoded_size': encoded_size,
    'process_time': process_time,
    'is_lossless': compression_accurate
}
def save_compressed(self, encoded_data: List[int], shape: Tuple[int,
int],
                    first_value: int, output_path: str):
    """Save compressed data to file."""
    # Save shape, first value, and encoded data
    data = {
        'shape': shape,
        'first_value': first_value,
        'encoded_data': encoded_data
    }

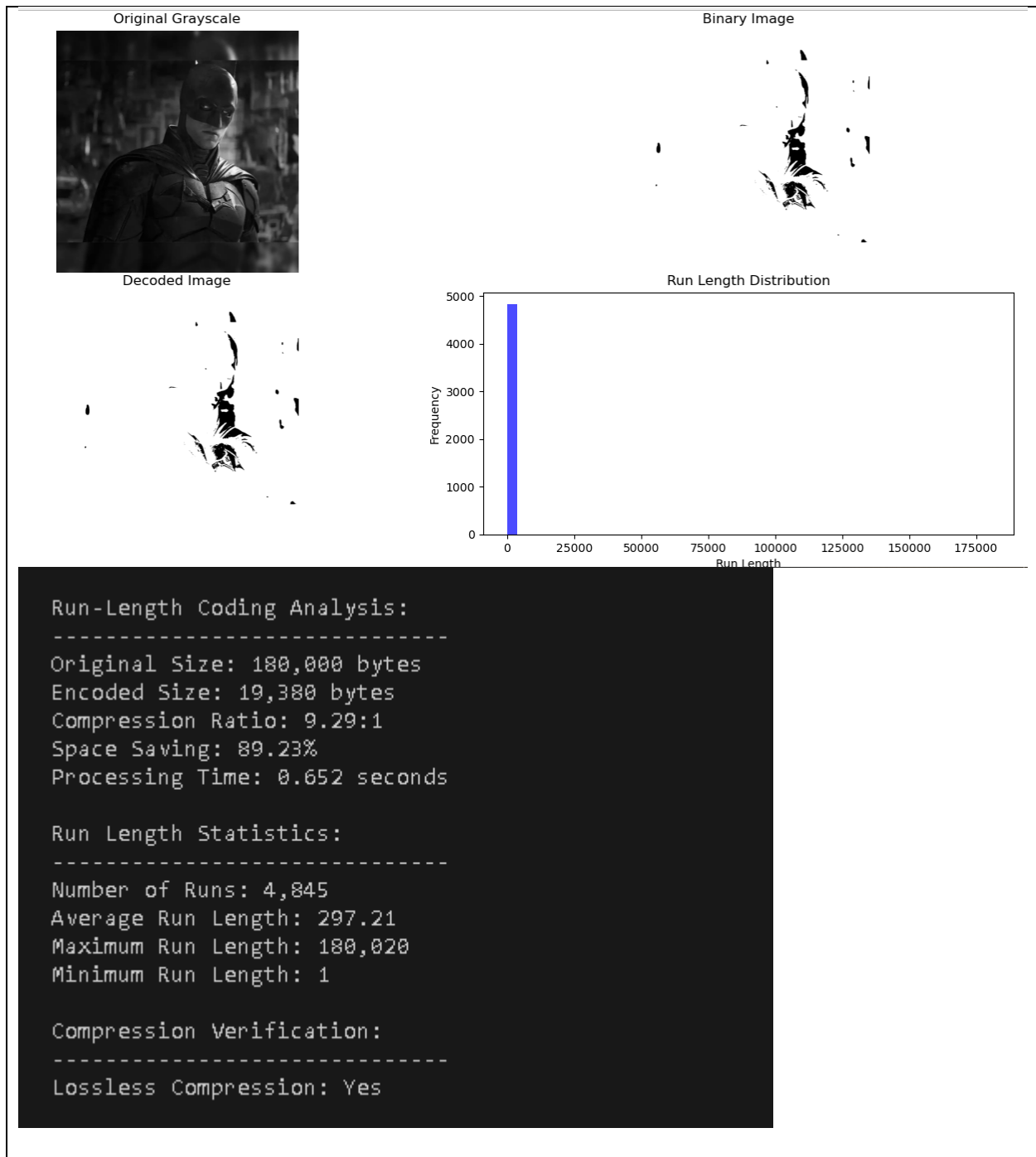
```

```

    np.save(output_path, data)
    def load_compressed(self, input_path: str) -> Tuple[np.ndarray,
dict]:
        """Load and decode compressed data from file."""
        # Load data
        data = np.load(input_path, allow_pickle=True).item()
        # Decode image
        decoded_image = self.run_length_decode(
            data['encoded_data'],
            data['shape'],
            data['first_value']
        )
        return decoded_image, data
# Example usage
if __name__ == "__main__":
    # Replace with your image path
    image_path = "Run-Length Coding.jpg"
    # Create compressor instance
    rlc = BinaryRLC(threshold=127)
    # Analyze compression
    results = rlc.analyze_compression(image_path)
    # Save compressed data
    binary_image = rlc.convert_to_binary(image_path)
    encoded_data = rlc.run_length_encode(binary_image)
    first_value = binary_image.flatten()[0]
    # Save and load example
    rlc.save_compressed(encoded_data, binary_image.shape,
first_value, "compressed.npy")
    decoded_image, saved_data =
rlc.load_compressed("compressed.npy")
    plt.show()

```

The results (Screenshot)



Conclusion

In conclusion, the Image Compression II proves to be the methodology within which the most efficient image compression techniques achieve high ratios of image file size to visual quality. It serves as a perfect example of balancing compression and fidelity, making the most of resources, both in storage size and in transmission bandwidth. This

significantly emphasizes the purpose of algorithms for specific case uses lossy versus lossless compressions. This is how yet another project buttresses the argument for significance in the data handling efficiency of today's digital age.